LNH-MusicTag 开发文档

版本：v1.5.3 | 仓库：github.com/lahuolou/lnh-musictag | 许可证：MIT v1.5.3

LNH-MusicTag 开发文档

版本：v1.5.3 | 仓库：github.com/lahuolou/lnh-musictag | 许可证：MIT

本文档与当前代码同步维护，覆盖架构、业务流程、数据模型、插件机制、API、部署、开发测试发布流程与安全运维。用于开发交接、日常维护、功能扩展与发布排错。

1. 项目概览与核心能力

LNH-MusicTag 是一款自托管音乐库标签管理工具。系统扫描本地音乐目录，读取和修改音频标签，通过多个在线数据源补全元数据，并提供去重、格式转换、乱码修复、简繁转换、音频内容识别等能力。

设计目标	实现方式	开发者需要关注
单二进制交付	Go 后端通过 go:embed 内嵌 Vue 3 构建产物	前端变更后需重新构建 web/dist，再编译后端
无 CGo	go-taglib（WASM 封装 TagLib）；modernc.org/sqlite	构建与部署尽量保持纯 Go 链路
插件化	刮削源、工具、下载器、服务协议统一注册	新插件优先遵守注册表和统一配置模型
后台任务	扫描、刮削、识别、转换、修复均异步执行	任务需要进度、结果统计、取消和刷新后恢复轮询
统一流水线	多个操作复用任务框架与公共实现	公共逻辑优先复用，避免同类功能行为不一致
多源并发	任何需要数据源的环节并发查询全部启用源，合并后挑最佳	新源只需实现 Source 接口，自动进入并发调度
阶段进度	任务按阶段上报 stageDone/stageTotal 与 hashDone/hashTotal	新增任务阶段必须上报阶段内进度，避免前端 0/N 假死

1.1 核心功能清单

音乐目录扫描：读取标签、探测音频属性、计算 SHA256，并将曲目索引持久化。

标签管理：编辑面板按配置渲染 **29 个元素**（17 可编辑 + 12 只读展示）；显示字段在「设置」页勾选并持久化，默认 12 个常用项。

多源并发刮削：按语言偏好并发查询全部启用源，合并结果后跨源挑最佳匹配（标题相似度 + 专辑完整性）再写入标签；批量/一键/歌词/海报/编辑面板搜索共用。

歌词多级兜底：网易云（短超时+重试）→ QQ → lrclib，不依赖搜索命中。

音频识别：Chromaprint 指纹 → AcoustID 查询 → MusicBrainz 补全；未命中时可尝试库内指纹匹配。

一键整理：可选去重（进回收站，跨卷已修复，**并发哈希可取消**），然后依次修复乱码、繁转简、补艺术家、执行刮削；进度条按当前阶段显示。

文件处理：格式转换、文件后缀修正、简繁转换、乱码修复。

任务与日志：异步进度（含阶段进度）、成功/失败/跳过统计、取消任务、操作日志；任务详情接口瘦身（只回失败项 + 计数）。

插件生态：内置刮削源插件组与工具插件组（FFmpeg/AcoustID），支持导入第三方 ZIP 插件并动态挂载 Web 面板；**同名重复导入 = 覆盖升级且保留已填配置**；插件卡片可折叠。

部署与更新：Docker Compose、FFmpeg 可选安装/卸载、多个更新源容错、单一版本源与 CI 校验。

2. 技术架构与模块边界

系统采用浏览器 SPA + Go HTTP 服务 + SQLite + 音乐文件目录的结构。服务默认监听 10248 端口。

层次	技术/组件	主要职责
前端	Vue 3、Vite、原生 CSS	页面、交互、任务轮询、国际化与主题
HTTP 入口	net/http、ServeMux（Go 1.22+ 路径参数）	路由注册、中间件、静态资源托管
身份与安全	auth.go、security.go	登录会话、CSRF 校验、请求体限制、登录限流
业务层	扫描、刮削、任务、识别、转换等 Go 模块	编排具体操作并统一返回任务状态
扩展层	plugin.go、plugins_api.go、plugin_runner.go、internal/scrape	插件注册、启停、配置持久化、多源调度、ZIP 安全导入与覆盖升级
存储层	SQLite、internal/store、tracks_db.go	应用设置、曲目索引与任务数据持久化
文件层	音乐目录、taglibx、FFmpeg	标签、封面、歌词、转码与文件重命名

2.1 关键模块索引

模块/路径	职责
main.go	程序入口、路由、中间件、静态资源、AppVersion（embed VERSION）、ensureHashes 并发哈希去重、/api/settings 编辑字段配置（editFields）
auth.go / security.go	会话鉴权、CSRF、请求体限制、登录失败限流
plugin.go / plugins_api.go / plugin_runner.go	插件注册表（registerPlugin/replacePlugin）、插件 API、启停和配置、ZIP 导入解压与安全校验、覆盖导入保留配置、面板静态路由
scrapejob.go	统一异步任务、进度（含阶段/哈希进度）、结果统计、取消、任务持久化与保留策略；一键刮削阶段编排；pickBest 跨源最佳匹配
pipeline.go	一键刮削的阶段编排与阶段内进度上报
identify.go	音频指纹识别与元数据补全

tracks_db.go / internal/store/config.go	曲目与配置数据持久化
internal/taglibx	音频标签、封面与歌词读写封装
internal/dedup / internal/finger / internal/audiofmt	去重、音频指纹与格式探测
internal/scrape	刮削源接口、17 个数据源实现、**SearchWithFallback 并发合并**、歌词多级兜底、MusicBrainz 客户端
convert.go / ffmpeg.go / renameclean.go	格式转换、FFmpeg 探测/安装/卸载与文件后缀修正
fixencoding.go / zhscript.go	乱码修复与简繁体转换
activity.go / version.go / static.go	操作日志、版本检查、静态资源缓存策略
web/src	Vue 页面、全局状态、API 封装、国际化和组件；**web/src/fields.js：29 元素字段定义与默认勾选**；**views/Plugin.vue：插件页（卡片折叠）**

2.2 请求与鉴权约定

登录、登出与当前会话接口为公开路由；其余业务 API 需要登录会话。

前端通过 web/src/api.js 统一封装 fetch，并发送 `X-Requested-With: LNH-MusicTag` 请求头。

业务请求统一经过鉴权、CSRF 校验和请求体大小限制等中间件。公开例外：

`/api/login` 豁免 CSRF（无会话可劫持）；

`/api/plugins/import` 豁免 8MiB 请求体上限（插件包上限 256MiB）；

第三方插件面板路由 `/api/plugins/{name}/panel` 额外放宽 `script-src 'unsafe-inline'`（仅该路由，详见第 9 章）。

错误响应统一为 `{"ok": false, "error": "..."}`；成功响应为对应业务 JSON（部分接口含 `ok: true`）。前端 api.js 读取 `data.error` 抛出异常。

前端没有使用 vue-router；页面由 view 与 activeTab 状态组合控制。

3. 关键业务流程

3.1 扫描流程

遍历配置的音乐目录，识别支持的音频扩展名。

读取现有标签，并探测时长、码率、采样率、声道等属性。

计算 SHA256，检测疑似乱码、繁体标签和待识别曲目。

解析缺失标签的文件名（例如"艺术家 - 标题"）。

以增量方式写入 SQLite，扫描进度通过 `/api/scan/status` 查询。

扫描顺序由 seq 记录，列表可在扫描完成后按列排序。

3.2 音频内容识别流程

音频文件 → 格式分流 → 必要时转 WAV → fpcalc/Chromaprint 指纹 → AcoustID → MusicBrainz → 写标签、抓歌词、刷新索引并按"艺术家 - 标题"重命名。

步骤	说明
格式处理	支持 14 种免转码格式（identifyExts），兼容已有 .mpc/.mka；其他格式可经 FFmpeg 转成 16-bit/44.1 kHz/双声道 WAV。
指纹生成	fpcalc -json；发送给 AcoustID 的应为压缩指纹；指纹库异步构建并持久化，增量更新。
在线查询	AcoustID 限速 3 QPS；MusicBrainz 限速 1 QPS。
回退策略	未配置 Key 或在线未命中时，可用库内指纹相似度匹配，阈值为 0.85。
最终处理	标题、艺术家、专辑、流派等统一转简体；写标签、抓歌词、刷新曲目索引并重命名。

3.3 一键刮削流程

阶段	操作	注意事项
0（可选）	自动去重	默认不勾选；重复文件移入配置目录 trash/ 回收站，不是直接永久删除；**哈希并发 8 线程 + 单文件 120s 超时兜底 + feeder/worker 双处检查取消**；进度按 hashDone/hashTotal 上报，不再"卡在 0/N"
1	乱码修复	与独立乱码修复 API 共用 repairTrackText；**textFields 覆盖全部 11 个文本元素（歌名/艺术家/专辑/专辑艺术家/风格/作曲家/作词/简介/歌词/语言/专辑类型）**
2	繁体转简体	通过 OpenCC t2s
3	补全文件名艺术家	文件名缺少艺术家时补齐
4	刮削标签、封面与歌词	多源并发查询（见 3.4）；刮削写回的标签统一转简体

阶段进度约定：任务进入每个阶段时重置 StageDone=0、StageTotal=当前曲目数；逐首处理前 StageDone++ 并更新 Current。前端进度条优先级：哈希未完成 → hashDone/hashTotal → 有阶段总数 → stageDone/stageTotal → 兜底 done/total。被前置去重移除的曲目仍会计入完成数，避免进度无法到达 total。取消任务通过 `/api/jobs/{id}/cancel` 发起，哈希循环与各阶段 worker 均即时响应。

3.4 多源并发刮削（v1.5.0 核心）

所有需要数据源的环节（批量刮削、一键刮削、歌词、海报、编辑面板搜索）统一走 `internal/scrape/sources.go` 的 `SearchWithFallback`：

确定查询集：指定源（若有）+ 语言优先链（中文曲目：国内源优先；外文曲目：国际源优先）中**所有启用**的源。

每个源独立 goroutine 并发请求（外部 API 均有超时与限速），结果合并返回。

`scrapejob.go` 的 `pickBest` 在合并结果上跨源比较：标题相似度 ≥ 0.35 中挑最高者；相似度并列时优先带专辑信息的命中。

将最佳匹配写入标签（歌名/艺术家/专辑/专辑艺术家/风格/年代/曲目号/封面/歌词等可选字段）。

`/api/scrape/search?source=auto` 走同一并发路径，前端编辑面板一次搜索即可看到全部启用源的命中（每条带来源徽标），供用户跨源对比。

3.5 去重删除（回收站 + 跨卷修复）

去重与列表删除均进回收站（配置目录 `trash/`，日志记 act_dedup/act_del）：

同卷：`os.Rename` 到 trash（8×1s 重试，处理 Windows 瞬时占用）。

跨卷（如 E:\mp32 → C:\config\trash）：`os.Rename` 必返回 EXDEV，改为 复制 → **立即关闭读句柄（defer Close 需提前触发，否则句柄阻塞删除）** → 删除源文件（8×1s 重试，兼容 Defender 实时扫描）。

历史 bug：跨卷一律失败（EXDEV 未处理）+ 跨卷 copy 分支 defer 句柄未提前释放 → 一键去重勾选后"移入回收站失败: file being used"。修复后删除耗时 16s → 39ms。

3.6 歌词链路（多级兜底）

`internal/scrape/lyrics.go` 的 FetchLyrics 按序尝试，**不依赖搜索是否命中**（搜索 0 命中时也按文件名解析出的标题/艺术家直接走链）：

网易云按 ID 拉取（独立 client，3s 短超时 ×3 重试，防挂起拖慢整链）；

网易未命中 → QQ 搜索歌词（提前到网易搜索之前，网易挂起时快速让位）；

QQ 搜索 → 网易搜索 → lrclib 兜底。

网易云歌词写入后统一转简体，避免"一键流水线先转、刮削后写回繁体覆盖"的问题。

3.7 封面加载缓存

后端封面接口带 **mtime 失效缓存**（coverCache：map[trackID]→{mtime, 封面数据}），文件未变更则复用缓存，并返回 `Cache-Control: private, max-age=3600`。

前端封面组件失败重试 2 次，onerror 静默隐藏。

实测：大列表重复请求 ~2ms，broken 封面 13 → 0。

3.8 任务状态与恢复

任务状态保存在内存任务表，并写入 SQLite `jobs` 表；进行中任务每 5 秒保存一次快照。

前端记录 jobId，登录或刷新后调用 resumeJobs() 重新挂载轮询。

任务记录可在进程重启后查询：已完成任务为 done，被进程中断的任务记录为 interrupted，恢复时同步写回库（不残留 running 状态）。

启动时执行保留策略：清理 30 天前的终态任务（running 任务不受影响）。

进度响应含 createdAt 与 duration（任务已耗时秒数），供前端与日志展示。

一键刮削/批量任务进度额外携带 `stageDone/stageTotal`（阶段进度）与 `hashDone/hashTotal`（去重哈希进度，哈希总量 >0 才出现），由 `jobProgressHandler` 统一透传。

自动断点续跑尚未实现；中断任务需要用户手动重新执行。

4. 数据模型与持久化

4.1 SQLite 表

表/数据	字段或内容	用途
settings	key/value；账号、bcrypt 密码哈希、session_secret、API Key、插件配置、edit_fields（编辑面板显示字段）等	持久化应用设置
tracks	id、path、data、seq	保存曲目 JSON、绝对路径和扫描顺序
jobs	id、kind、total、done、current、status、stage、stage_label、hash_done、hash_total、stage_done、stage_total、results（JSON）、created_at、updated_at	任务状态持久化与重启后查询；创建/取消/完成即时写库，运行中每 5 秒快照
activity_logs	操作日志条目	登录、任务、删除、插件等关键操作留痕，支持删除/清空

曲目 ID 定义为 SHA256 的前 16 个字符；曲目数据主体以完整 JSON 保存于 tracks.data。启动时加载曲目，外部已删除的文件会从索引中清理。扫描、编辑、重命名和删除等操作会同步更新数据库。数据库使用 WAL + busy_timeout(5000)，连接池上限 4。

4.2 Track 主要字段

字段	含义
ID / Path / FileName / Ext	稳定 ID（SHA256 前 16 字符）、服务器绝对路径、文件名和扩展名
Size / Duration	文件大小、时长
SHA256	用于内容哈希去重
Bitrate / SampleRate / Channels / BitDepth	音频属性
HasCover / HasLrcFile / HasLyrics	是否有内嵌封面、是否存在外挂歌词文件、是否已内嵌歌词
HasTrad / Garbled / NeedsIdentify	繁体、疑似乱码、待音频识别标记
Tags	按标准键扁平化保存的标签映射（可编辑字段键，见 4.3）

4.3 编辑面板字段清单（29 个元素）

字段定义位于 `web/src/fields.js`（`FIELDS`），后端 `main.go` 的 `editFieldsList()` 解析设置页勾选的逗号串，`defaultEditFields` 为默认 12 项；设置存于 settings 表 `edit_fields` 键。

可编辑字段（17 个，前端经 TAG_KEYS 映射到标签键写入）： 歌名、艺术家、专辑、专辑艺术家、风格、年份、曲目号、总曲目数、作曲家、作词、语言、专辑类型、简介（COMMENT）、光盘编号、总光盘数、歌词（独立 textarea 区块）。

只读展示字段（12 个，`.ro-val` 渲染）： 封面、文件名、外挂 LRC、比特率、采样率、位深度、声道、时长、文件大小、SHA256、路径、MusicBrainz 录音/专辑（取自 tags.MUSICBRAINZ_*）。

默认勾选 12 项（DEFAULT_EDIT_FIELDS）： 封面、歌名、艺术家、专辑、专辑艺术家、风格、年份、曲目号、总曲目数、作曲家、作词、歌词。

> 说明：v1.4.10 时代"28 个可勾选字段"指批量面板的字段勾选；v1.5.0 编辑面板字段为 29 个元素（17 可编辑 + 12 只读），批量面板字段勾选仍为 28 字段 + 封面。

4.4 数据一致性注意事项

标签写入、重命名、文件删除和曲目索引更新应保持一致；优先复用统一刷新函数 refreshTrack。

文件路径必须先规范化，并确认仍处于允许操作的音乐目录内。

去重与删除操作使用回收站策略（配置目录 trash/，同卷 rename、跨卷复制后删源）。

新增字段时需同步考虑 JSON 兼容、旧数据库兼容、前端展示和测试覆盖。

5. 插件扩展机制

插件注册表统一管理刮削源、工具、下载器与服务协议。内置插件按组展示：刮削源插件组（17 个数据源 + 自定义源）、工具插件组（FFmpeg、AcoustID）。第三方插件组支持导入解耦设计的 ZIP 插件。

5.1 插件属性

属性	用途
Name	唯一标识，例如 scrape.qq、tool.ffmpeg
Label / Kind	前端显示名称与插件类型（scrape/tool/download/protocol）
Enabled / Builtin	启用状态；内置插件（含 auto）不可停用/删除/覆盖
Installable	是否支持一键安装
NeedsKey / KeyField / KeyConfigured	外部 Key 依赖、对应配置键与是否已配置
Fields / Config	前端动态表单定义与插件配置；Config 序列化时至少为空对象 {}
Panel	第三方插件 Web 面板声明（plugin.json 中 "panel": "xxx.html"），宿主自动挂载为插件 tab

5.2 第三方插件（解耦设计）

导入：上传 ZIP → 校验 → 解压到配置目录 plugins/<name>/ → 注册（`replacePlugin`）。

覆盖导入：同名插件重复导入 = 覆盖升级——解压前 `os.RemoveAll` 旧目录、重新解压新包；注册时保留旧插件 `Config`（用户已填的 Key/Cookie 等）与注册顺序，仅更新定义（Label/Version/Fields/Hint/Entries）。**内置插件（Builtin=true）拒绝覆盖**（400"内置插件不可覆盖"）。JSON 声明导入同规则：命中已注册且非内置 → 覆盖并保留 Config。

安全约束：压缩包 ≤256MiB、单文件 ≤256MiB、解压总量 ≤512MiB（防压缩炸弹）；拒绝符号链接/设备/命名管道条目（防符号链接逃逸）；解压目标必须位于插件目录内（防 zip-slip）；当前平台入口与 Web 面板声明文件必须存在。

面板：plugin.json 声明 "panel" 后，前端自动在编辑/批量/去重旁生成插件 tab，通过同源 iframe 加载 `/api/plugins/{name}/panel`；面板可调用 `/api/plugins/{name}/run` 执行插件入口（op 分发），并通过 `/api/fs/scan-dir` 做通用目录扫描。

前端卡片：第三方插件（ext）卡片**配置区默认折叠**，点卡片头展开/收起（▲▼）；开关与删除按钮常驻。

卸载：删除配置与解压目录；前端列表页刷新后 tab 消失。

宿主不侵入插件内部实现，插件逻辑全部在 ZIP 内，宿主只提供注册、运行、面板与通用文件接口。

5.3 新增插件建议流程

实现所需的统一接口（plugin.json + 入口），明确输入、输出、超时和错误处理。

在初始化流程中注册插件，定义唯一 Name、Kind、配置字段与启停回调。

确保配置可持久化，旧配置缺字段时也能安全加载（覆盖导入时保留旧 Config）。

验证插件快照、前端表单、启停状态、Key 脱敏和异常场景。

若插件包含 Web 面板，必须评估同源 iframe 与 CSP 放宽带来的安全风险（见 9.3）。

6. API 分类索引

除公开例外外，业务 API 需要登录会话，并遵守 CSRF 请求头约定。错误响应统一为 `{"ok":false,"error":"..."}`。

类别	方法与路径	用途
会话	POST /api/login	登录并下发会话 Cookie（豁免 CSRF）
会话	POST /api/logout；GET /api/me；POST /api/change-password	登出、读取会话、修改密码
扫描	POST /api/scan；POST /api/scan/pause；GET /api/scan/status	开始扫描、暂停/继续、查询进度
扫描	GET /api/tracks/refresh	强制刷新曲目索引
曲目	GET /api/tracks；POST /api/tracks/delete	查询曲目（含筛选）、批量删除（进回收站，记 act_del）
曲目	GET /api/track/{id}	单曲完整详情（含歌词正文，编辑面板按需拉取，列表不再下发）

曲目	GET /api/tracks/{id}/cover；POST /api/tracks/{id}/cover	读取（mtime 缓存 + Cache-Control）/ 写入封面
曲目	GET /api/tracks/{id}/lrcfile；GET /api/lyrics	外挂 LRC 文件、歌词查询（多级兜底）
标签	POST /api/tracks/{id}/tags	写标签
刮削	GET /api/scrape/sources；GET /api/scrape/search	读取源列表、搜索指定源（source=auto 并发合并）
刮削	POST /api/tracks/{id}/scrape；POST /api/scrape/batch；POST /api/scrape/onekey	单曲、批量、一键刮削
任务	GET /api/jobs；GET /api/jobs/{id}；POST /api/jobs/{id}/cancel	进行中列表、查询进度（**瘦身：只回失败项 + summary 计数 + createdAt/duration + 阶段/哈希进度**）、取消任务
任务	GET /api/scrape/jobs/{id}	批量刮削进度（兼容旧路径）
工具	POST /api/convert；POST /api/fix-encoding；POST /api/convert-script	格式转换、乱码修复、简繁转换
识别	POST /api/identify；POST /api/rename-clean	音频识别、修正文件后缀
去重	GET /api/duplicates；POST /api/duplicates/remove	获取重复分组（`?log=1` 才记操作日志，防轮询刷屏）、移除选中文件（进回收站，记 act_dedup）
设置	GET/POST /api/settings	读取/保存应用设置（GET 返回 editFields 数组；POST 接受 EditFields 存 edit_fields 键）
插件	GET/POST /api/plugins；POST /api/plugins/import；DELETE /api/plugins/{name}	列表/更新、导入 ZIP/JSON（**同名覆盖保留配置**）、删除插件
插件	POST /api/plugins/{name}/run；GET /api/plugins/{name}/panel；GET /api/plugins/{name}/panel/{path...}	执行插件入口、插件面板与面板静态资源
插件	POST /api/fs/scan-dir	第三方插件面板通用目录扫描
FFmpeg	GET /api/ffmpeg/status；POST /api/ffmpeg/install；POST /api/ffmpeg/uninstall	检测、安装、卸载 FFmpeg
日志	GET /api/logs；POST /api/logs/delete；POST /api/logs/clear	操作日志读取、删除条目、清空
版本	GET /api/version；GET /api/version/check	读取缓存版本状态、强制实时检查

7. 部署与配置

7.1 Docker Compose 配置要点

推荐使用 ghcr.io/lahuolou/lnh-musictag:latest，容器端口为 10248。需要挂载音乐目录和配置卷。

```
services:
  lnh-musictag:
    image: ghcr.io/lahuolou/lnh-musictag:latest
    container_name: lnh-musictag
    ports:
      - "10248:10248"
    volumes:
      - /your/music/path:/music
      - lnh_config:/config
    environment:
      - LNH_ADMIN_USER=admin
      - LNH_ADMIN_PASS=
      - ACOUSTID_API_KEY=
      - LNH_INSTALL_FFMPEG=false
    restart: unless-stopped

volumes:
  lnh_config:
```

提示：以上为结构示例。部署时请将 /your/music/path 替换为真实路径，并按实际需要配置密钥；不要将密钥提交到公开仓库。

7.2 主要环境变量

变量	作用/注意事项
LNH_ADMIN_USER / LNH_ADMIN_PASS	首次初始化账号密码；后续账号信息保存在数据库中
LNH_INSTALL_FFMPEG	设为 true/1/yes/on 时，启动前尝试安装 FFmpeg
LNH_CONFIG_DIR	配置目录（数据库/插件/回收站），非容器默认 /config
LNH_PORT	HTTP 监听地址；容器内固定使用 10248
ACOUSTID_API_KEY 等	各数据源 API Key 的初始化配置

7.3 构建命令

```
cd web && npm ci && npm run build
cd .. && go vet ./...
go test ./...
go build -ldflags "-s -w" -o lnh-musictag .
```

Windows 本机调试：`Start-Process .\lnh-musictag.exe -ArgumentList "-port","10248" -WindowStyle Hidden`。运行中 exe 被占用时需先 `Stop-Process -Name lnh-musictag -Force` 再构建。

8. 开发、测试与发布流程

8.1 日常开发建议流程

先定位功能所属模块，优先复用公共函数和任务框架。
修改后运行 gofmt，并检查前后端构建结果。
新增或变更 API 时同步更新前端封装、开发文档和测试。
执行 go vet ./...、go test ./... 与 go build。
在隔离的测试配置目录启动服务，冒烟验证登录、曲目列表、编辑、批量任务和插件页。
检查浏览器控制台、服务日志、失败统计和文件/数据库一致性。

8.2 发布流程

完成代码与前端构建，并通过静态检查、单元测试和构建。
执行本地 UI 冒烟测试，确认关键页面和浏览器控制台正常。
复核 CSRF、XSS、SQL 参数化查询、路径穿越与第三方插件风险。
同步发布副本并提交代码，推送 main 分支触发 Docker 镜像构建。
只修改根目录 VERSION 作为版本唯一来源，并确保版本号与 Git tag 一致（CI version-check 强制校验）。
创建带 v 前缀的 Release tag，检查 CI 校验结果。
打包源码时确认包含 web/dist、Dockerfile、Compose 与 entrypoint 文件。
按流程：用户确认前不发布；先交付核验，再执行正式发布。

8.3 发布前检查表

前端已重新构建，web/dist 内容完整。
go vet ./...、go test ./...、go build 均通过。
VERSION 与 Release tag 完全一致。
登录、扫描、编辑、批量刮削、取消任务、插件配置（含覆盖导入/折叠）和更新检查通过冒烟测试。
密钥无明文泄漏；默认/测试账号未进入生产配置。
Docker 镜像、数据卷、FFmpeg 选项与部署说明已核对。
发布说明、变更记录和回滚方式已准备。

9. 安全与运维注意事项

9.1 安全措施

风险面	措施
账号与会话	bcrypt 密码哈希；随机 session_secret；HttpOnly、SameSite Cookie；登录失败限流
CSRF	校验 X-Requested-With，自定义请求头并结合 SameSite；/api/login 豁免
XSS	Vue 默认转义；标签文本写入前净化；CSP script-src 'self'（宿主页）
SQL 注入	database/sql 参数化查询
路径穿越	filepath.Clean 并校验路径位于音乐目录/插件目录内；插件 ZIP 解压防 zip-slip
请求滥用	请求体限制（业务 8MiB，插件导入豁免）；外部 API 有超时与限速
插件面板	仅插件面板路由放宽 script-src 'unsafe-inline'；iframe sandbox 去掉 allow-forms
删除安全	列表删除与去重删除均移入回收站 trash/，不直接永久删除
插件覆盖	仅非内置插件可被覆盖导入；内置插件（scrape.*/FFmpeg/AcoustID）拒绝覆盖与删除
公网部署	建议置于反向代理后并启用 HTTPS

9.2 中间件例外汇总

`/api/login`：豁免 CSRF。

`/api/plugins/import`：豁免 8MiB 请求体上限（插件包可达 256MiB）。

`/api/plugins/{name}/panel` 与 `/api/plugins/{name}/panel/{path...}`：CSP 允许内联脚本（兼容第三方插件面板）。

9.3 第三方插件安全边界

插件面板是同源 iframe；当前以"仅面板路由放宽 CSP"作为兼容手段，不建议全站放宽。

长期加固方向：面板外链脚本化（禁止内联 <script>/onclick）、nonce/hash 替代 unsafe-inline、独立源 + sandbox + postMessage 鉴权转发。

ZIP 导入已做三重上限与符号链接/路径穿越防护；覆盖导入只更新声明定义、不信任旧目录残留；安装前核对插件来源。

9.4 故障排查速查

现象	优先检查
更新后页面仍旧	浏览器/代理缓存；强制刷新；确认 index.html no-cache
更新检查总是显示最新	VERSION 文件与 Release tag 是否同步
任务重启后中断	查询 jobId 历史状态（interrupted）；当前不支持自动断点续跑

一键刮削卡在去重 0/N	旧版本问题（v1.5.1 修复：并发哈希+可取消+单文件超时）；升级镜像并强刷页面
进度条不显示阶段	任务未进入阶段（如仅批量刮削）；或前端缓存旧 JS，强刷
扫描较慢	首次全量 SHA256 计算可能耗时
刮削源报 500/TLS	上游接口变化、网络限制或源服务故障
需 Key 的源无结果	插件页 Key 状态及设置页配置
无法转换音频	FFmpeg 是否安装、路径是否正确、编码器是否可用
MP3 转换提示 Unknown encoder	检查 FFmpeg 是否包含 libmp3lame 编码器
重复导入插件提示已存在	旧版本行为；v1.5.2 起同名覆盖导入（保留配置），内置插件仍拒绝
歌词/作词/语言等元素乱码或繁体未修复	v1.5.3 起乱码修复与繁转简覆盖全部 11 个文本元素；深层双重损坏（GBK→单字节→再 Unicode 化，如 `ÈÄÐÔ³»,Ä`）信息已部分丢失，无法完整还原原字
去重后文件不见	检查配置目录下 trash/ 回收站
接口报错格式异常	确认错误响应为 {"ok":false,"error":...} 统一结构

10. 版本记录

v1.5.3：乱码修复与简繁转换**覆盖全部 11 个文本元素**（新增作词/语言/专辑类型；繁体检测 detectTraditional 与阶段简繁转换共用同一 textFields 清单）；`fixMojibake` 修复——Latin-1 范围内 GBK 字节恰好构成合法 UTF-8 时旧逻辑直接重组会误改（如"黄家驹"GBK 字节），现 UTF-8 重组与 GBK 解码均作为候选参与得分比较取最优；一键刮削面板移除与下方进度条重复的独立阶段标签；新增回归测试（GBK-合法 UTF-8 不误改、11 字段全修复且 ASCII 不误动）。

v1.5.2：插件页两项体验——第三方插件卡片可折叠（配置区默认收起）；重复导入同名插件改为**覆盖升级且保留已填配置**（ZIP 与 JSON 导入同规则，内置插件仍拒绝覆盖）；插件页间距优化；插件开发说明文案同步更新（覆盖语义）。

v1.5.1：一键刮削自动去重卡死修复——哈希改为 8 线程并发 + 单文件 120s 超时兜底 + feeder/worker 双处取消检查 + hashDone/hashTotal 进度上报（前端哈希进度条）；取消任务即时生效。

v1.5.0：多源并发刮削（SearchWithFallback 并发合并 + pickBest 跨源最佳）；编辑面板 29 元素 + 设置页字段配置（默认 12 常用）；去重删除跨卷修复（EXDEV + 句柄释放 + 重试，一键去重可用）；歌词多级兜底（网易 3s 短超时 / QQ 优先 / 脱离搜索命中限制 / 刮削结果统一转简体）；任务接口瘦身（/api/jobs/{id} 只回失败项，/api/track/{id} 单曲详情）；封面 mtime 缓存 + Cache-Control；筛选拆分（有/无歌词、有/无海报）；批量海报写入修复；清除调试残留。

v1.4.10：29 字段模型（28 可勾选 + 勾选列）、一键刮削 4 阶段、封面优化、手机适配、第三方插件解耦框架、操作日志增强。

v1.4.10（P0 系列，已随 v1.5.0 发布）：命名统一 lnh-musictag；CSP 精确放宽（仅面板路由）；插件 ZIP 安全加固；删除进回收站；任务状态持久化与恢复写回库；版本单一源 + CI 校验；代码精简轮。

开发文档优化项（已落地）：API 错误响应统一 {ok:false,error}；jobs 保留策略（30 天清理）；任务 createdAt/duration 可观测；任务恢复/耗时/保留策略单元测试。

—— 文档结束 ——